

Lab 4: Indexing Strategies — Every Index Type That Matters (70 min)

Pairs with the [Schema Patterns Playbook](#). Part C is Playbook #6 (Hot/Cold Split); Part D is Playbook #1 applied at the secondary-index level.

Learning Objectives

By the end of this lab you will be able to:

- Build secondary, covering (`STORING`), partial, hash-sharded, expression, and inverted (JSONB) indexes
- Use `EXPLAIN` and `EXPLAIN ANALYZE` to confirm which index a query uses and why
- Quantify the cost an *unused* or *misused* index imposes on writes
- Eliminate per-row primary-key lookups with `STORING`
- Read `crdb_internal.table_indexes` and `[SHOW INDEXES FROM]` to audit a table's index inventory
- Drop redundant indexes safely

Prerequisites

- Running 3-node cluster (Lab 3's still works, or start a fresh one)

Setup

```
scripts/crdb up           # start the 3-node cluster (skip if it is already running)
scripts/crdb sql          # open a SQL shell
```

Everything runs in Docker — see [Lab 1](#) for the cluster layout. Use `scripts/crdb.sh` on macOS, Linux, and **WSL** — WSL is Linux, so the shell script is the right one there. Only a native Windows shell (`cmd` or PowerShell) needs `scripts\crdb.bat` .

```

CREATE DATABASE shop;
USE shop;

CREATE TABLE customers (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email   STRING UNIQUE NOT NULL,
  name    STRING NOT NULL,
  joined  TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE orders (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id   UUID NOT NULL REFERENCES customers(id),
  status        STRING NOT NULL CHECK (status IN ('open','paid','shipped','cancelled')),
  total         DECIMAL(12,2) NOT NULL CHECK (total >= 0),
  region        STRING NOT NULL,
  payload       JSONB NOT NULL DEFAULT '{}',
  placed        TIMESTAMPTZ DEFAULT now()
);

INSERT INTO customers (email, name)
SELECT 'user' || g || '@example.com', 'User ' || g
FROM generate_series(1, 1000) g;

INSERT INTO orders (customer_id, status, total, region, payload)
SELECT
  (SELECT id FROM customers ORDER BY random() LIMIT 1),
  (ARRAY['open','paid','shipped','shipped','shipped','cancelled'])[1 + (random()*5)::INT],
  (random() * 500)::DECIMAL(12,2),
  (ARRAY['us-east','us-west','eu-west'])[1 + (random()*2)::INT],
  jsonb_build_object(
    'channel', (ARRAY['web','mobile','api'])[1 + (random()*2)::INT],
    'tags',    (ARRAY['fast','urgent','gift','plain'])[1 + (random()*3)::INT]
  )
FROM generate_series(1, 50000);

CREATE STATISTICS s1 FROM customers;
CREATE STATISTICS s2 FROM orders;

```

Tasks

Part A: Confirm Pain Before Indexing (10 min)

1. Run the natural "orders for a customer" query without an index:

```

EXPLAIN ANALYZE
SELECT id, status, total
FROM orders
WHERE customer_id = (SELECT id FROM customers LIMIT 1);

```

Look in the output for:

- **scan node** — which table/index?

- **spans** — is it [-] (whole keyspace) or a narrow key range?
- **actual row count** at the scan vs the final result — how much work is thrown away?

2. Confirm it's a full scan:

```
EXPLAIN SELECT id, status, total FROM orders
WHERE customer_id = (SELECT id FROM customers LIMIT 1);
```

The `scan` node should reference `orders@primary` and span the whole table.

3. Add a basic secondary index:

```
CREATE INDEX orders_by_customer ON orders(customer_id);
```

`customer_id` is a foreign key, and it had no index until now. **CockroachDB does not index FK columns automatically** (neither does PostgreSQL). Every column in a `REFERENCES` clause is a join column, and an unindexed join column means a full scan of the child table on every join — the most common slow join in practice. Index them by default.

4. Re-run EXPLAIN ANALYZE. Confirm:

- Scan is now `orders@orders_by_customer`
- Span is single-key ([/<customer_id> - /<customer_id>])
- Actual scan-row-count ≈ final result row count
- **KV time drops by an order of magnitude.**

Part B: STORING — Avoid the Per-Row PK Lookup (10 min)

The query reads `id`, `status`, `total`. The index above only contains `customer_id`, `id`. To get `status` and `total`, executor does a PK lookup per row — visible as an **index join** in the plan.

1. See the index join:

```
EXPLAIN SELECT id, status, total FROM orders
WHERE customer_id = (SELECT id FROM customers LIMIT 1);
```

2. Rebuild with STORING:

```
DROP INDEX orders_by_customer;
CREATE INDEX orders_by_customer
ON orders(customer_id)
STORING (status, total);
```

3. Re-run EXPLAIN. No more index join — the index has everything the query needs.

4. Measure the write cost. Time a `STATUS = 'shipped'` mass-update with and without storing. With `STORING`, every status change rewrites both the PK row and the index entry:

```
\set show_times
UPDATE orders SET status = 'shipped' WHERE status = 'paid';
```

STORING is a tradeoff — fast reads, slower writes. Use it on hot read paths, not blindly.

Part C: Partial Index (*Playbook #6 — Hot/Cold Split*) (10 min)

This is the **Hot/Cold Split** pattern from the Playbook: build a tiny index over only the hot subset (here, `status = 'open'`) and leave the cold archive un-indexed.

Most orders are `shipped`. A dashboard showing "open orders only" can skip the rest.

1. Run the open-orders query:

```
EXPLAIN ANALYZE
SELECT id, customer_id, total
FROM orders
WHERE status = 'open'
ORDER BY total DESC
LIMIT 20;
```

Without a targeted index this is a full table scan + sort.

2. Create a partial index — only open orders, sorted:

```
CREATE INDEX orders_open
ON orders(total DESC)
STORING (customer_id)
WHERE status = 'open';
```

3. **Re-run EXPLAIN ANALYZE.** Plan reads only the partial index (~8000 rows), pre-sorted. `LIMIT 20` becomes a tiny range scan.

4. What if a row changes status?

```
UPDATE orders SET status = 'paid'
WHERE id = (SELECT id FROM orders WHERE status = 'open' LIMIT 1);
```

CockroachDB removes the row from the partial index in the same transaction. Verify:

```
SELECT count(*) FROM orders WHERE status = 'open';
```

Part D: Hash-Sharded Secondary Index for Time Series (*Playbook #1 on an index*) (10 min)

The `placed` column is a write timestamp — perfectly monotonic. A naïve index on it hotspots.

1. Try a naïve index:

```
CREATE INDEX orders_placed_naive ON orders(placed);

EXPLAIN ANALYZE
SELECT id, customer_id, total
FROM orders
WHERE placed > now() - INTERVAL '5 minutes';
```

2. Inspect the index ranges:

```
SHOW RANGES FROM INDEX orders@orders_placed_naive;
```

Likely a small number of ranges with the rightmost edge handling all current writes.

3. Replace with hash-sharded:

```
DROP INDEX orders_placed_naive;

CREATE INDEX orders_placed_hashed
ON orders(placed)
USING HASH
STORING (customer_id, total)
WITH (bucket_count = 8);
```

⚠ **Clause order is fixed and unforgiving:** `ON t(col) → USING HASH → STORING (...) → WITH (bucket_count = N)`. Both of these are syntax errors:

```
... ON orders(placed) STORING (customer_id, total) USING HASH WITH (bucket_count = 8);
... ON orders(placed) USING HASH WITH (bucket_count = 8) STORING (customer_id, total);
```

Worth knowing because a failed `CREATE INDEX` is easy to miss: the next `EXPLAIN` still returns a plan — just not the one you thought you were testing.

4. Confirm spread:

```
SHOW RANGES FROM INDEX orders@orders_placed_hashed;
```

5. **Re-run EXPLAIN ANALYZE.** The plan now scans all 8 buckets in parallel — slightly more work per range, but no hotspot.

Part E: Expression Indexes — Index Computed Expressions (10 min)

You frequently search by lowercased email. Don't add a `lower_email` column — index the expression directly.

1. Add an expression index:

```
CREATE INDEX customers_email_lower
ON customers ((lower(email)));
```

2. Use it:

```
EXPLAIN
SELECT id, name FROM customers
WHERE lower(email) = 'user42@example.com';
```

The plan should reference `customers_email_lower` with a single-key span.

3. What if you write `email = 'User42@Example.com'` instead?

```
EXPLAIN
SELECT id, name FROM customers
WHERE email = 'User42@Example.com';
```

This uses the original UNIQUE index, *not* the expression index, because the optimizer can't prove `email = 'X'` is equivalent to `lower(email) = lower('X')` without case-folding both. Expression indexes only help queries that use the *same* expression.

Part F: Inverted Index for JSONB (10 min)

Our `orders.payload` is JSONB. A query like "find all orders with `channel = 'mobile'`" can't use a regular index unless we GIN-index the JSONB column.

1. Try without an index:

```
EXPLAIN ANALYZE
SELECT id, total FROM orders
WHERE payload @> '{"channel": "mobile"}'::JSONB
LIMIT 100;
```

Full table scan + per-row predicate evaluation.

2. Add the inverted index:

```
CREATE INVERTED INDEX orders_payload_idx ON orders(payload);
```

3. Re-run EXPLAIN ANALYZE. The plan now uses `orders_payload_idx`, which stores one entry per path in the JSONB tree.

4. What about array-valued queries?

```
-- Find orders tagged 'gift'
EXPLAIN ANALYZE
SELECT id FROM orders
WHERE payload @> '{"tags": "gift"}'::JSONB;
```

This also uses the inverted index.

Part G: Auditing & Cleanup (10 min)

1. List every index on the `orders` table:

```
SHOW INDEXES FROM orders;
```

2. From the system catalog:

```
-- Qualify every column: table_indexes and index_columns share several names
-- (index_name, descriptor_name), so bare references are ambiguous.
SELECT ti.index_name, ic.column_name, ic.column_direction, ic.column_type
FROM crdb_internal.table_indexes ti
JOIN crdb_internal.index_columns ic USING (descriptor_id, index_id)
WHERE ti.descriptor_name = 'orders'
ORDER BY ti.index_name, ic.column_name;
```

3. Which indexes have NEVER been used?

```
SELECT index_name, total_reads, last_read
FROM crdb_internal.index_usage_statistics ius
JOIN crdb_internal.table_indexes ti
  ON ius.table_id = ti.descriptor_id AND ius.index_id = ti.index_id
WHERE ti.descriptor_name = 'orders'
ORDER BY total_reads;
```

Indexes with `total_reads = 0` after a workload has been running for a while are good drop candidates.

4. Drop the naïve index (we replaced it with hash-sharded in Part D, but it's still gone — verify):

```
SHOW INDEXES FROM orders;
```

5. Quick check of total table size including indexes:

```
SELECT sum(range_size_mb) AS total_mb
FROM [SHOW RANGES FROM TABLE orders WITH DETAILS];
```

Cleanup

```
DROP DATABASE shop CASCADE;
```

The cluster keeps running between labs — that is the point of it being persistent. To wipe everything and start fresh at any time:

```
scripts/crdb reset
```

Lab 4 Deliverables

□ **Six index types built and tested:** secondary, covering (STORING), partial, hash-sharded, expression, inverted □ **Plans verified:** confirmed via EXPLAIN ANALYZE which index each query used □ **STORING tradeoff measured:** removed an index join from a hot read path, observed the write cost □ **Index inventory audited:** `SHOW INDEXES`, `crdb_internal.table_indexes`, `index_usage_statistics`

Challenge Exercises

1. **Dashboard tune.** This query runs every 30 seconds:

```
SELECT region, status, count(*), sum(total)
FROM orders
WHERE placed > now() - INTERVAL '1 hour'
GROUP BY region, status;
```

Design **one** index that minimizes its cost. Justify column order, STORING (or not), and hash sharding (or not). Measure before and after.

2. **Composite vs separate.** Compare these two designs:

```
-- Design A: one composite
CREATE INDEX a ON orders(customer_id, status, placed);

-- Design B: two narrower
CREATE INDEX b1 ON orders(customer_id);
CREATE INDEX b2 ON orders(status);
```

For which queries does A win, and for which does B? Where do they tie?

3. **Drop ALL indexes** on `orders`, then run the workload. How does latency change vs. all indexes present? Which is the single most-impactful index, by latency improvement when re-added?

Reference

Index type	Use case	Notes
Secondary	Equality / range on non-PK column	The default.
Covering / STORING	Hot read path needs extra cols	Slower writes.
Partial	Predicate-filtered subset	Massive size savings when most rows don't match.
Hash-sharded	Monotonic key, write hotspot	Use <code>bucket_count</code> $\approx$ concurrency.
Expression	Index a function of a column	Query must use the <i>same</i> expression.
Inverted (GIN)	JSONB, arrays	One entry per JSON path.
Vector (vector(N))	Embeddings (v23.2+)	Out of scope for this course.